Ejercicio 1. Método signo

1. Análisis del método

- **Qué hace:** Determina si un número es positivo (1), negativo (-1) o cero (0).
- **Entradas:** Un número entero x.
- **Salidas:** 1 si x > 0, -1 si x < 0, y 0 si x = 0.

2. Complejidad Ciclomática

El código presenta dos estructuras de decisión (if y else if).

- **Cálculo:** M = 2 \text{ (predicados)} + 1 = 3.
- Se requieren **3 tests** para cubrir todas las ramas.

3. Diseño de casos de prueba

ID	Descripción	Entrada	Esperado	Rama
TC01	Número positivo	5	1	x > 0
TC02	Número negativo	-3	-1	x < 0
TC03	Cero	0	0	else

4. Implementación en JUnit

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class SignoTest {
   @Test
   void testSignoPositivo() {
      assertEquals(1, Signo.signo(5));
   }
   @Test
   void testSignoNegativo() {
      assertEquals(-1, Signo.signo(-3));
   }
   @Test
   void testSignoCero() {
      assertEquals(0, Signo.signo(0));
   }
}
```

Ejercicio 2. Método clasificarEdad

1. Análisis del método

- **Qué hace:** Clasifica una edad en categorías (Infancia, Niñez, etc.) o lanza excepción si es negativa.
- **Casos límite:** 0, 6, 12, 18, 25, 60

2. Complejidad Ciclomática

Contamos los predicados: edad < 0, edad < 6, edad < 12, edad < 18, edad < 25, edad < 60.

- **Cálculo:** M = 6 \text{ (predicados)} + 1 = 7.

3. Diseño de casos de prueba (Selección)

ID	Descripción	Entrada	Esperado
TC01	Edad inválida	-1	IllegalArgumentException
TC02	Límite inferior Infancia	0	"Infancia"
TC03	Límite superior Infancia	5	"Infancia"
TC04	Límite Niñez	6	"Niñez"
TC05	Límite Vejez	60	"Vejez"

4. Implementación en JUnit

```
@Test
void testEdadInvalida() {
   assertThrows(IllegalArgumentException.class, () ->
ClasificarEdad.clasificarEdad(-1));
}

@Test
void testInfancia() {
   assertEquals("Infancia", ClasificarEdad.clasificarEdad(0));
}
```

Ejercicio 3. Método contarPositivos

1. Análisis del método

- **Qué hace:** Recorre un array y cuenta cuántos elementos son estrictamente mayores a cero.

2. Complejidad Ciclomática

- Estructuras: Un bucle for y un if dentro.
- **Cálculo:** M = 2 + 1 = 3.

3. Diseño de casos de prueba

ID	Descripción	Entrada	Esperado
TC01	Array vacío	{}	0
TC02	Todos negativos	{-1, -2}	0
TC03	Mezcla con ceros	{0, 1, -1, 2}	2

4. Implementación en JUnit

```
@Test
void testContarPositivos() {
   assertEquals(0, Ejercicios.contarPositivos(new int[]{}));
   assertEquals(2, Ejercicios.contarPositivos(new int[]{1, -2, 0, 3}));
}
```

Ejercicio 4. Método calificacion

1. Análisis del método

- **Qué hace:** Convierte una nota numérica en texto usando un switch.
- **Entradas:** Entero nota.
- **Salidas:** "SUSPENSO", "SUFICIENTE", etc. o excepción si está fuera de rango.

2. Complejidad Ciclomática

- **Cálculo:** Validación inicial (||) y las ramas del switch (5 casos).
- **Resultado:** V(G) = 2 \text{ (validación)} + 4 \text{ (n-1 casos del switch)} = 6.

3. Diseño de casos de prueba

ID	Entrada	Esperado	Rama
TC01	11	Exception	Fuera de rango
TC02	3	"SUSPENSO"	Case 0-4
TC03	5	"SUFICIENTE"	Case 5
TC04	6	"BIEN"	Case 6
TC05	8	"NOTABLE"	Case 7-8
TC06	10	"SOBRESALIENTE"	Default

4. Implementación en JUnit

```
@ParameterizedTest
@CsvSource({"3, SUSPENSO", "5, SUFICIENTE", "6, BIEN", "8, NOTABLE", "10, SOBRESALIENTE"})
```

```
void testCalificaciones(int nota, String esperado) {
   assertEquals(esperado, Ejercicios.calificacion(nota));
}
```

Ejercicio 5. Método esBisiesto

1. Análisis del método

- **Qué hace:** Determina si un año es bisiesto basándose en las reglas del calendario gregoriano.
- **Entradas:** Un número entero anyo.
- **Salidas:** Un valor booleano (true si es bisiesto, false si no).
- **Reglas:** Es bisiesto si es divisible por 4 y no por 100, O si es divisible por 400.

2. Complejidad Ciclomática

Utilizando la fórmula M = P + 1 (donde P es el número de predicados):

- **Predicados:** anyo % 4 == 0, anyo % 100 != 0, anyo % 400 == 0.
- **Cálculo:** 3 + 1 = 4.
- **Mínimo de tests:** 4 casos para cubrir todas las decisiones lógicas.

3. Diseño de casos de prueba

ID	Descripción	Entrada	Esperado	Rama/Condición
TC01	Divisible por 4 y no por 100	2024	true	Divisible 4
TC02	Divisible por 100 pero no por 400	1900	false	Divisible 100
TC03	Divisible por 400	2000	true	Divisible 400
TC04	No divisible por 4	2023	false	No divisible 4

4. Implementación en Java y JUnit

- Clase EsBisiesto.java

```
public class EsBisiesto {
   public static boolean esBisiesto(int anyo) {
      return (anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0);
   }
}
```

- Clase EsBisiestoTest.java

Java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class EsBisiestoTest {
   @Test
   void testEsBisiesto() {
      assertTrue(EsBisiesto.esBisiesto(2024), "2024 debería ser bisiesto");
      assertFalse(EsBisiesto.esBisiesto(1900), "1900 no debería ser bisiesto");
      assertTrue(EsBisiesto.esBisiesto(2000), "2000 debería ser bisiesto");
      assertFalse(EsBisiesto.esBisiesto(2023), "2023 no debería ser bisiesto");
   }
}
```

Ejercicio 6. Implementar Piedra - Papel - Tijera

1. Análisis del método

- **Qué hace:** Simula el juego entre dos jugadores y determina el ganador o si hay empate.
- **Entradas:** Dos Strings, jugador1 y jugador2 ("PIEDRA", "PAPEL" o "TIJERA").
- **Salidas:** Un String con el resultado ("EMPATE", "GANA J1", "GANA J2") o una excepción si la entrada no es válida.

2. Complejidad Ciclomática

Para una implementación estándar con validación:

- **Predicados:** 1 (Validación) + 1 (Empate) + 3 (Combinaciones de victoria de J1).
- **Cálculo:** 5 + 1 = 6.
- **Mínimo de tests:** Al menos 6 casos para cubrir error, empate, las 3 formas de ganar de J1 y el caso por defecto (gana J2).

3. Diseño de casos de prueba

ID	Descripción	Entrada 1	Entrada 2	Esperado
TC01	Jugada inválida	"FUEGO"	"PIEDRA"	IllegalArgumentException
TC02	Empate	"PAPEL"	"PAPEL"	"EMPATE"
TC03	Piedra gana a Tijera	"PIEDRA"	"TIJERA"	"GANA J1"
TC04	Tijera gana a Papel	"TIJERA"	"PAPEL"	"GANA J1"

TC05	Papel gana a Piedra	"PAPEL"	"PIEDRA"	"GANA J1"
TC06	J2 gana a J1	"PIEDRA"	"PAPEL"	"GANA J2"

4. Implementación en Java y JUnit

- Clase PiedraPapelTijera.java

```
public class PiedraPapelTijera {
  public static String jugar(String jugador1, String jugador2) {
    // Validación
    if (!esValida(jugador1) || !esValida(jugador2)) {
      throw new IllegalArgumentException("Jugada no válida"); [cite: 192]
    }

    if (jugador1.equals(jugador2)) return "EMPATE"; [cite: 191]

    if ((jugador1.equals("PIEDRA") && jugador2.equals("TIJERA")) ||
      (jugador1.equals("TIJERA") && jugador2.equals("PAPEL")) ||
      (jugador1.equals("PAPEL") && jugador2.equals("PIEDRA"))) {
      return "GANA J1";
    }

    return "GANA J2";
  }

  private static boolean esValida(String jugada) {
    return jugada.equals("PIEDRA") || jugada.equals("PAPEL") ||
jugada.equals("TIJERA"); [cite: 182]
  }
}
```

- Clase PiedraPapelTijeraTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
class PiedraPapelTijeraTest {
  @Test
  void testJugadaInvalida() {
    assertThrows(IllegalArgumentException.class, () ->
PiedraPapelTijera.jugar("FUEGO", "PIEDRA")); [cite: 49]
  }

  @Test
  void testEmpate() {
    assertEquals("EMPATE", PiedraPapelTijera.jugar("PIEDRA", "PIEDRA")); [cite:
49]
  }

  @Test
  void testGanaJ1() {
    assertEquals("GANA J1", PiedraPapelTijera.jugar("PIEDRA", "TIJERA"));
    assertEquals("GANA J1", PiedraPapelTijera.jugar("TIJERA", "PAPEL"));
    assertEquals("GANA J1", PiedraPapelTijera.jugar("PAPEL", "PIEDRA"));
  }

  @Test
  void testGanaJ2() {
    assertEquals("GANA J2", PiedraPapelTijera.jugar("PIEDRA", "PAPEL"));
  }
}
```

Ejercicio 7. Validador de contraseñas

1. Análisis e Implementación

- **Puntos:** +1 por cada criterio (largo, Mayús, Minús, Núm, Símbolo).
- **Resultado:** 0-2 Débil, 3-4 Media, 5 Fuerte.

2. Complejidad Ciclomática

- **Cálculo:** 5 verificaciones de caracteres + 2 de rango de puntos.
- **Resultado:** V(G) \approx 8.

3. Diseño de tests

- TC01: "123" (1 pt) \rightarrow "DEBIL".
- TC02: "Password123" (4 pts: len, May, Min, Num) \rightarrow "MEDIA".
- TC03: "P@ssw0rd1" (5 pts) \rightarrow "FUERTE".

4. Implementación en JUnit

```
@Test
void testPassword() {
   assertEquals("DEBIL", Ejercicios.evaluarPassword("123"));
   assertEquals("FUERTE", Ejercicios.evaluarPassword("P@ssw0rd1"));
}
```